

# Milestone 1 Evaluation — Covid-19 Detection

## 1. Project Title, Team Members

**Project Title:** Covid-19 Detection

- Drew Quashie (dquashie2024@my.fit.edu)
- Amanda Ogbonna (kogbonna2025@my.fit.edu)
- Loleyi Oluwatomisin (ooluwatomisi2023@my.fit.edu)
- Richard Alonso Garcia (ralonsogarci2023@my.fit.edu)

## 2. Faculty Advisor

**Name:** Dr. Zahra Nematzadeh **Email:** znematzadeh@fit.edu

## 3. Client

Dr. Zahra Nematzadeh (znematzadeh@fit.edu) — Faculty Advisor, Florida Tech

## 4. Progress of current Milestone (progress matrix)

| Task                                                          | Completion % | Drew Quashie | Amanda Ogbonna | Loleyi Oluwatomisin | Richard Alonso Garcia | To do |
|---------------------------------------------------------------|--------------|--------------|----------------|---------------------|-----------------------|-------|
| Requirements & Project Plan Documentation                     | 100%         | 10%          | 70%            | 10%                 | 10%                   | N/A   |
| CoughVid Audio Data Preprocessing & MFCC/Spectrogram Pipeline | 100%         | 70%          | 10%            | 10%                 | 10%                   | N/A   |
| Audio Augmentation Pipeline (Noise, Pitch & Time Shifts)      | 100%         | 70%          | 10%            | 10%                 | 10%                   | N/A   |
| Baseline PyTorch CNN Architecture Development                 | 100%         | 10%          | 10%            | 70%                 | 10%                   | N/A   |
| React Audio Recording Interface Setup                         | 100%         | 10%          | 10%            | 10%                 | 70%                   | N/A   |

## 5. Discussion (at least a paragraph) of each accomplished task (and obstacles) for the current Milestone

### Requirements & Project Plan Documentation

The planning and documentation set for the project is complete. `project_plan.md` defines the title, team, goals, approach, algorithms/tools, technical challenges, and the three-milestone schedule. The Software Requirements Specification (SRS v1.0, IEEE Std 830) specifies 16 functional requirements (SFR1–SFR16) and 10 non-functional requirements (SNFR1–SNFR10), covering the guided three-cough capture, input validation, XGBoost cough gating, MFCC/log-Mel feature extraction, CNN inference with majority-vote aggregation, Low/Medium/High risk output, and on-device IndexedDB storage. The Software Design Document (SDD v1.0) maps these to four modules (audio capture, ML inference, results display, data management) with component and interface design. The Software Test Documentation (IEEE Std 829-2008) maps every SFR and SNFR to a test case. The main obstacle here was scope discipline: pinning down the requirements early enough that the ML pipeline design and the browser constraints (stateless server, on-device storage) could be fixed before implementation began.

### Literature Review & Research Documentation

A literature review was conducted to establish the research foundation for the project and prepare for the eventual publication of our work. Research focused primarily on previous COVID-19 detection methods using cough and respiratory audio to understand related machine-learning approaches. Each paper was documented by its dataset and size, preprocessing and feature-extraction methods, machine learning or deep learning models, evaluation metrics, and final results. Particular attention was given to approaches using MFCCs, Mel-spectrograms, CNNs, and other audio-classification techniques so that previous methods could be compared with the direction of our own system. The findings were organized into a research table and supporting notes that can later contribute to the literature review and methodology sections of the final paper. A notable obstacle was the large variation between studies in datasets, sample sizes, preprocessing techniques, model architectures, and reported metrics, which made direct comparisons difficult and required careful review to ensure that only the most relevant information was documented.

### CoughVid Audio Data Preprocessing & MFCC/Spectrogram Pipeline

The CoughVid dataset preprocessing is complete.

`project_notebooks/modules/preprocessing/dataframe_process.py` filters the CoughVid metadata with polars, and `wav_converter.py` converts all of the dataset's webm recordings to wav in parallel using ffmpeg. The core feature pipeline lives in `project_notebooks/modules/preprocessing/audio_process.py`: `segment_cough / segment_cough_robust` extract cough events, `compute_SNR` scores signal quality, and `MelSpectrogramPipeline`, `MFCCPipeline`, and `ComombinedSpectMFCCPipeline` produce fixed-length Mel, MFCC, and joint MFCC + log-Mel features respectively. `data_preprocess.ipynb` runs and tests the load → segment → feature-extraction flow, and

`wav_length_exp.ipynb` studied cough lengths to justify fixed 1-second segments (98.5th percentile). The primary obstacle was file-format heterogeneity: CoughVid ships mostly webm with variable sample rates and durations, which required the `ffmpeg` setup helper (`ffmpeg_fix.py`), robust segmentation, and fixed-length enforcement before batching.

### **Audio Augmentation Pipeline (Noise, Pitch & Time Shifts)**

Augmentation is completed which is used to counter dataset class imbalance and background-noise variability (two of the technical challenges named in the project plan). SpecAugment time masking and frequency masking are implemented in `audio_process.py`, with the experiment notes recording time stretching and masking choices from the Phase 1 experiments. Additive noise and pitch/time shifting are specified in the project plan and SDD and are being wired into the training-time data pipeline. The obstacle is balancing augmentation strength: aggressive masking or shifting risks destroying cough cues that the classifier depends on, so augmentation parameters are being validated against the 1-second segmentation decisions from the length study.

### **Baseline PyTorch CNN Architecture Development**

The model code structure exists but training has not started (pending). `project_notebooks/modules/nn/base_nn.py` defines `BaselineCNN`, a convolutional classifier over the three classes (Healthy, Symptomatic, COVID-19), and `base_nn.py` provides the reusable `ConvBlockV1` (`Conv2d` → `BatchNorm` → `ReLU`) plus a `DefaultCNN` with a `LazyLinear` head. The intended training entry point, `neural_network.ipynb`, is still a stub, and `feature_extraction.ipynb` is empty — the immediate obstacle is sequencing: training cannot begin in earnest until the preprocessing and augmentation pipelines (above) produce consistent fixed-length feature tensors.

### **React Audio Recording Interface Setup**

This task has been completed. The project website itself is live — a static site (`project_notebooks/project_website/`) deployed by GitHub Pages workflow to <https://covidwav.io>, containing the project plan page and shared styles — but the React component that records three guided coughs through the Web Audio API has not been built yet. The obstacle is browser-side constraints identified in the SRS/SDD: microphone permission handling, format/duration/SNR/clipping validation before upload, and keeping health history in IndexedDB so the server stays stateless.

## **6. Discussion (at least a paragraph) of contribution of each team member to the current Milestone**

### **Drew Quashie**

I created the data preprocessing and feature extraction pipelines along with the data augmentation. Our base augmentation uses PyTorch time stretching, time masking and frequency masking to

help diversify our dataset. Additionally, to statistically verify what our set input length should be we did a statistical experiment using a histogram of the lengths of each segmented cough. From this we saw a heavily right skewed sample space and as such we used a cut off of the longest 98.5 percentile and below of the lengths of the coughs which rounded off to 1s. the MFCC and Mel spectrogram data was generated with pytorch modules which will be used as transform classes in the coming Dataset class for training. We had some issues with ffmpeg and differing audio formats of which I fixed with some scripts I added.

### **Amanda Ogbonna**

For this milestone, my focus was primarily on the research and documentation side of the project. I conducted research of previous COVID-19 detection studies, focusing mainly on cough and respiratory audio classification. I reviewed and documented the datasets and dataset sizes, preprocessing and feature-extraction techniques, machine learning/deep learning models, evaluation metrics, results, and any limitations reported in each study. I organized this information into a detailed notes section as well as a concise research table for quick lookup so that we can compare previous approaches with our own project. We also plan on publishing a paper upon our project's completion, so documenting all resources and any outside information used is very important. I also looked closely at studies using MFCCs, Mel-spectrograms, CNNs, and similar audio-preprocessing methods because these approaches directly relate to the pipeline our team will eventually develop. This research will continue throughout the project as we refine our methodology and prepare documentation needed for publication.

### **Loleyi Oluwatomisin**

I mainly worked on creating and testing our baseline CNN model. I made a convolutional block that consisted of 1 Convolution layer, a Batch norm layer and a rectified linear unit layer (ReLU). This block is the main workhorse of our default CNN model. This CNN model then flattened to go through 2 dense layers that used ReLU activations and our final layer consisted of 3 logits and a Sigmoid Activation function to keep our outputs in a range of 0 -> 1. This allows a confidence output the 3 classes of health, covid-19 and symptomatic.

### **Richard Alonso Garcia**

I built the audio recording feature for our web app, which captures the cough samples our CNN and quality check model rely on. I used the Web Audio API with an AudioWorkletNode instead of the browser's built-in MediaRecorder, since MediaRecorder produces a different audio format depending on the browser (Chrome and Firefox output webm, while Safari outputs mp4), which would have introduced inconsistency into the exact data our models need to analyze. My implementation captures raw audio directly from the microphone, resamples it to a fixed 16kHz rate regardless of the user's device, and encodes it into a consistent WAV format before it's sent to our backend. I also built a guided three-cough capture flow, where the user is prompted to cough three separate times, with a live microphone level meter to confirm the mic is picking up sound and on screen indicator dots that fill in as each cough is detected, so recording stops automatically once all three are captured. I handled edge cases like denied microphone permissions, unsupported browsers, and missed or false cough detections so the recorder failures

are caught instead of leaving the user stuck. I also designed a wireframe for our sign in screen and main dashboard, laying out the recording flow, risk/result display, and cough tracking chart so we can have a better idea of our approach before our implementation.

## 7. Plan for the next Milestone (task matrix) — Milestone 2 (October 26th).

| Task                                                                                                | Drew Quashie | Amanda Ogbonna | Loleyi Oluwatomisin | Richard Alonso Garcia |
|-----------------------------------------------------------------------------------------------------|--------------|----------------|---------------------|-----------------------|
| Train & fine-tune advanced CNN models and transfer learning architectures (e.g., ResNet)            | x            | x              |                     |                       |
| Complete initial model evaluation and generate performance reports (F1 Score, ROC-AUC)              | x            |                | x                   |                       |
| Develop FastAPI backend endpoints for handling client audio uploads and running inference pipelines | x            | x              | x                   |                       |
| Build initial React web UI components for displaying prediction results and risk feedback           |              |                |                     | x                     |

## 8. Discussion (at least a paragraph) of each planned task for the next Milestone

### Train & fine-tune advanced CNN models and transfer learning architectures (e.g., ResNet)

With the preprocessing pipeline producing fixed-length joint MFCC + log-Mel tensors, Milestone 2 moves from the untrained `DefaultCNN` in `base_cnn.py` to real training runs. The plan is to first establish a baseline with the existing convolutional architecture, then fine-tune a ResNet transfer-learning variant (listed in the Algorithms and Tools table of `project_plan.md`). The SDD's algorithm-design section defines the training and label-engineering approach (three-logit sigmoid outputs for Healthy/Symptomatic/COVID-19), and the augmentation pipeline built in Milestone 1 (SpecAugment, noise, pitch/time shifts) plus class-imbalance handling will be applied during training. Notebook `neural_network.ipynb` is the intended entry point.

### Complete initial model evaluation and generate performance reports (F1 Score, ROC-AUC)

Evaluation will follow the metrics already committed to in the project plan: accuracy, precision, recall, F1 score, and ROC-AUC. The test documentation defines the expected performance targets the reports must be checked against, including end-to-end latency under 6 seconds and

inference under 1.5 seconds, alongside per-class effectiveness. Deliverables are performance reports over the held-out CoughVid split, with results summarized for the Milestone 2 review and carried into the final paper.

### **Develop FastAPI backend endpoints for handling client audio uploads and running inference pipelines**

The backend will expose the analysis endpoint exercised by the test plan (`POST /v1/analyze`): accept uploaded cough audio, run the XGBoost cough gate, segmentation, feature extraction, and CNN inference, then return the risk assessment with confidence and vote breakdown. The SDD's application-logic layer specifies this flow, and the design keeps the server stateless — no health history is stored server-side (IndexedDB only on the client), which is a stated SNFR. Python and FastAPI are the committed stack from the project plan's tool table.

### **Build initial React web UI components for displaying prediction results and risk feedback**

Building on the recording interface work, Milestone 2 adds the results-display components: rendering the Low/Medium/High risk level with confidence, vote breakdown, clinical guidance (e.g., self-isolation vs. contacting a healthcare provider), and the required disclaimers, per the SRS functional requirements and the SDD's human-interface design screens. These components consume the FastAPI response and feed the on-device IndexedDB store that backs the longitudinal tracking dashboard planned for Milestone 3.

## **11. Date(s) of meeting(s) with Faculty Advisor during the current milestone**

1. Tuesday August 25th 11:15am — Olin Engineering Complex room 353
2. Thursday September 10th 11:45am — Olin Engineering Complex room 353
3. Thursday September 17th 11:45am — Olin Engineering Complex room 353
4. Thursday September 24th 11:45am — Olin Engineering Complex room 353

## **12. Faculty Advisor feedback on each task for the current Milestone**

**Requirements & Project Plan Documentation:**

**Literature Review and Research Documentation:**

**CoughVid Audio Data Preprocessing & MFCC/Spectrogram Pipeline:**

**Audio Augmentation Pipeline (Noise, Pitch & Time Shifts):**

**Baseline PyTorch CNN Architecture Development:**

**React Audio Recording Interface Setup:**

**Faculty Advisor Signature:** \_\_\_\_\_

**Date:** \_\_\_\_\_

## 13. Evaluation by Faculty Advisor

*Return this page to Dr. Chan (HC 209) or email scores to [pkc@cs.fit.edu](mailto:pkc@cs.fit.edu).*

**Score (0–10) for each member** — circle one score; two adjacent scores may be circled for a .25 increment, or write any real number between 0 and 10.

### 1. Drew Quashie

0 1 2 3 4 5 5.5 6 6.5 7 7.5 8 8.5 9 9.5 10

**Score:** \_\_\_\_\_

### 2. Amanda Ogbonna

0 1 2 3 4 5 5.5 6 6.5 7 7.5 8 8.5 9 9.5 10

**Score:** \_\_\_\_\_

### 3. Loleyi Oluwatomisin

0 1 2 3 4 5 5.5 6 6.5 7 7.5 8 8.5 9 9.5 10

**Score:** \_\_\_\_\_

### 4. Richard Alonso Garcia

0 1 2 3 4 5 5.5 6 6.5 7 7.5 8 8.5 9 9.5 10

**Score:** \_\_\_\_\_

**Faculty Advisor Signature:** \_\_\_\_\_

**Date:** \_\_\_\_\_